

Platformă CI/CD pentru o aplicație de inventar auto

Implementarea unui flux DevOps complet pe infrastructură AWS

Documentație proiect final

Cursant: Andrei-Alexandru Praf

CUPRINS

CAPITOLUL 1. Obiectivele proiectului	3
1.1 Obiective principale	3
1.2 Practici DevOps vizate	3
CAPITOLUL 2. Arhitectura soluției.....	4
2.1 Componentele infrastructurii.....	4
Diagrama de ansamblu a arhitecturii	4
2.2 Integrarea pipeline-urilor în ciclul de dezvoltare (SDLC)	5
Fluxul complet vizual.....	5
2.3 Gestionarea stării mediului live	5
Capitolul 3. Tehnologii utilizate.....	6
Capitolul 4. Implementarea pipeline-urilor.....	7
4.1 Pipeline 1 - Validare (CI).....	7
Mecanism de declanșare	7
Logica de execuție	7
4.2 Pipeline 2 - Staging	8
Mecanism de declanșare	8
Logica de execuție	8
4.3 Pipeline 3 - Producție (Blue/Green)	9
Mecanism de declanșare	9
Logica de execuție	9
Strategia Blue/Green și baza de date partajată.....	10
4.4 Pipeline 4 - Monitorizare și auto-recuperare.....	11
Mecanism de declanșare	11
Logica de execuție	11
ANEXE.....	13
Anexa 1 - Infrastructura ca și cod (Terraform)	13
Anexa 2 - Reguli de securitate și IP fix	13
Anexa 3 - Verificarea unui release nou (Jenkinsfile.production)	13
Anexa 4 - Comutarea traficului Blue/Green	14
Anexa 5 - Dovezi vizuale suplimentare.....	14

CAPITOLUL 1. Obiectivele proiectului

Scopul acestui proiect a fost construirea unei platforme complete de integrare și livrare continuă (CI/CD) pentru o aplicație web de tip inventar auto, plecând de la codul sursă și ajungând până la un mediu de producție funcțional, găzduit pe infrastructură cloud. Am pornit de la ideea că un proces de livrare software modern nu ar trebui să depindă de pași manuali repetitivi, ci de pipeline-uri automate care validează, testează și implementează codul într-un mod predictibil și sigur.

Aplicația de bază este un sistem simplu de gestiune a unui parc auto, structurat pe trei niveluri: frontend scris în React, backend în FastAPI și o bază de date PostgreSQL. Deși aplicația în sine este relativ simplă, obiectivul real al proiectului nu a fost aplicația, ci întreaga infrastructură și fluxul DevOps.

1.1 Obiective principale

- Automatizarea completă a procesului de build, testare și implementare, astfel încât o modificare de cod să poată ajunge în producție fără intervenție manuală în etapele intermediare.
- Asigurarea fiabilității prin testare automată la fiecare pull request și prin verificări de sănătate (health checks) după fiecare implementare.
- Implementarea unei strategii de deployment fără downtime, prin mecanismul Blue/Green, care permite trecerea traficului de la o versiune la alta fără întreruperea serviciului.
- Găzduirea întregii soluții pe AWS, cu infrastructura definită ca și cod (Infrastructure as Code) prin Terraform, pentru ca întregul sistem să fie reproductibil.
- Introducerea unui mecanism de auto-recuperare (self-healing), care detectează containerele nefuncționale și le repornește automat.
- Implementarea unui proces formal de release, cu aprobare manuală înainte de orice implementare în producție.

1.2 Practici DevOps vizate

Pe parcursul proiectului am urmărit să aplic practici concrete din ecosistemul DevOps, nu doar la nivel teoretic. Printre acestea se numără: separarea clară a mediilor (staging și producție), gestionarea configurațiilor sensibile prin credențiale securizate, versionarea infrastructurii în git și izolarea componentelor prin containerizare.

CAPITOLUL 2. Arhitectura soluției

Soluția este construită în jurul a trei mașini virtuale dedicate, găzduite pe AWS EC2, fiecare cu un rol bine definit. În locul unei singure instanțe care rulează totul, am ales să izolez complet serverul de automatizare, mediul de staging și mediul de producție. Astfel, o problemă într-un mediu nu afectează celelalte medii, iar resursele fiecărei mașini sunt dedicate unui singur scop.

2.1 Componentele infrastructurii

- VM-ul Jenkins - serverul central de automatizare. Rulează Jenkins într-un container Docker și orchestrează toate pipeline-urile. Nu găzduiește aplicația în sine; rolul lui este să construiască imaginile, să le publice și să coordoneze implementările pe celelalte mașini.
- VM-ul Staging - mediul de testare. Primește implementări automate la fiecare merge în ramura principală din GIT și permite verificarea aplicației înainte ca aceasta să ajungă în producție.
- VM-ul Production - mediul de producție. Găzduiește baza de date de producție partajată împreună cu cele două medii Blue și Green ale aplicației, între care se comută traficul la fiecare implementare.

Diagrama de ansamblu a arhitecturii

AWS EC2 - eu-central-1

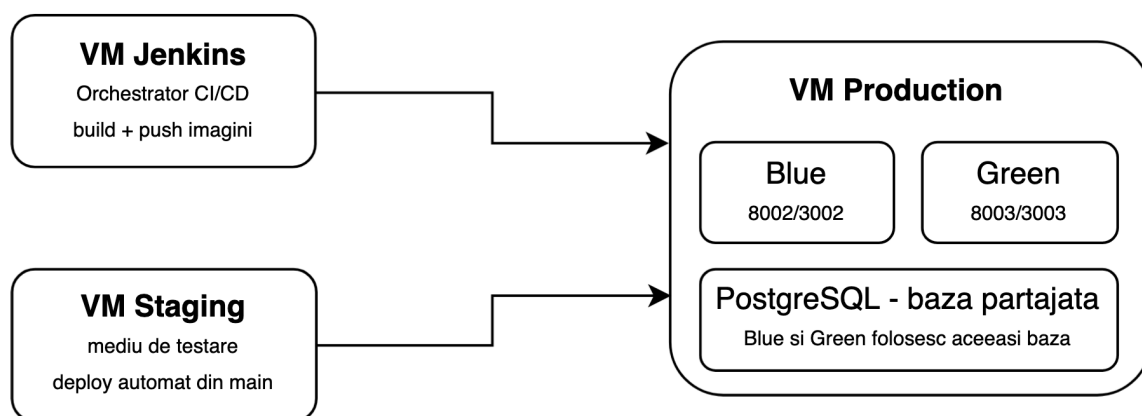


Figura 1: Arhitectura celor trei VM-uri și fluxul de deployment prin SSH

2.2 Integrarea pipeline-urilor în ciclul de dezvoltare (SDLC)

Fluxul complet urmărește ciclul de viață al dezvoltării software. Fiecare etapă a dezvoltării este acoperită de un pipeline dedicat, iar tranziția de la o etapă la alta se face prin mecanisme de bine definite.

1. Dezvoltatorul lucrează pe o ramură de funcționalitate și deschide un pull request către ramura main.
2. Pipeline-ul de validare (CI) se declanșează automat, rulează lint și teste unitare. Merge-ul este blocat până când toate verificările trec.
3. După merge-ul în main, pipeline-ul de staging construiește imaginile, le publică și implementează aplicația în mediul de staging, unde rulează teste de integrare.
4. Când versiunea din staging este considerată gata, se deschide un pull request din main către ramura release. Acest pull request necesită aprobare manuală.
5. După aprobare și merge în release, pipeline-ul de producție se declanșează și efectuează implementarea Blue/Green cu comutare de trafic fără downtime.

Fluxul complet vizual

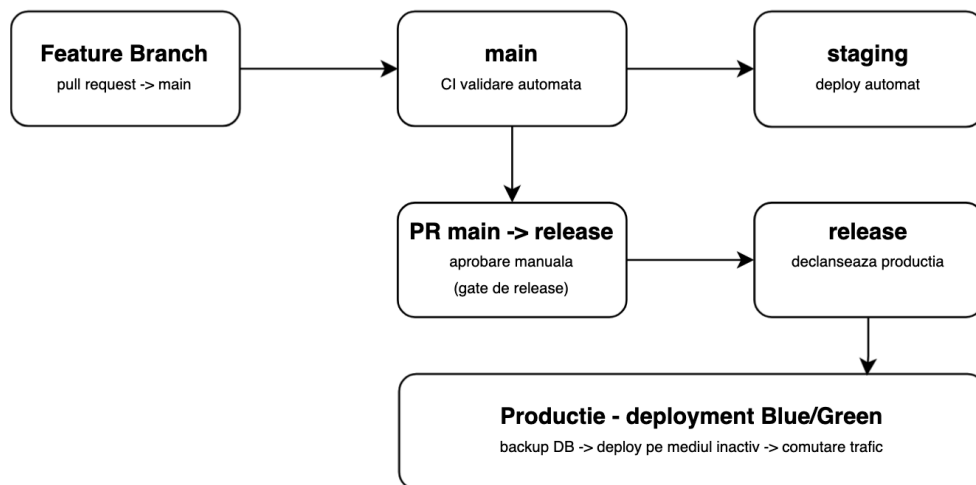


Figura 2: Fluxul de livrare, de la feature branch până la producția Blue/Green

2.3 Gestionarea stării mediului live

Un aspect important al arhitecturii este modul în care sistemul știe care mediu (Blue sau Green) este activ la un moment dat. Am ales să folosesc fișierul de configurare Nginx stocat direct pe mașina de producție, într-o locație persistentă, în afara git-ului și în afara workspace-ului Jenkins. La fiecare implementare, pipeline-ul actualizează acest fișier cu portul mediului nou devenit activ. Astfel, fișierul reflectă întotdeauna starea reală curentă a sistemului, iar atât pipeline-ul de producție cât și cel de monitorizare citesc din aceeași sursă.

Capitolul 3. Tehnologii utilizate

Pentru fiecare parte a proiectului am ales tehnologii pe care le-am folosit în trecut sau le-am întâlnit în timpul cursului și care sunt răspândite în industrie. Am mers pe tool-uri cu documentație bună și comunitate mare, ca să pot găsi rapid soluții când apar probleme.

Tehnologie	Rol în proiect și motivul alegerii
Jenkins	Serverul de automatizare care rulează toate pipeline-urile CI/CD. Ales pentru controlul total asupra procesului și pentru posibilitatea de a rula direct comenzi Docker.
Docker & Docker Compose	Containerizarea întregului stack aplicativ. Asigură că aplicația rulează identic indiferent de mediul gazdă și simplifică orchestrarea componentelor.
Terraform	Definirea infrastructurii ca și cod. Permite crearea celor 3 VM-uri, a regulilor de securitate și a adreselor IP fixe cu o singură comandă, complet reproductibil.
AWS EC2	Găzduirea cloud a celor 3 mașini virtuale. Am folosit instanțe t3.small în regiunea Frankfurt (eu-central-1).
PostgreSQL	Baza de date relațională, atât pentru staging cât și pentru producție. În producție, o singură instanță este partajată între Blue și Green.
GitHub & GHCR	Găzduirea codului, gestionarea pull request-urilor, regulile de protecție a ramurilor și registrul de imagini Docker (GitHub Container Registry).
React / FastAPI	Frontend-ul, respectiv backend-ul aplicației de inventar auto.

Capitolul 4. Implementarea pipeline-urilor

Proiectul cuprinde patru pipeline-uri Jenkins. Trei dintre ele acoperă etapele principale ale livrării (validare, staging, producție), iar al patrulea asigură monitorizarea continuă și auto-recuperarea. În cele ce urmează descriu fiecare pipeline din punct de vedere al mecanismului de declanșare, al logicii de execuție și, în cazul producției, al strategiei de deployment.

4.1 Pipeline 1 - Validare (CI)

Mecanism de declanșare

Acest pipeline este configurat ca Multibranch Pipeline și se declanșează automat la deschiderea unui pull request către ramura main. Strategia de descoperire a ramurilor este setată să construiască doar ramurile care au un pull request deschis, evitând astfel construirea inutilă a tuturor ramurilor din repository.

Logica de execuție

- Checkout - descărcarea codului din ramura pull request-ului.
- Verificări backend - lint cu flake8 și black, urmate de testele unitare cu pytest.
- Verificări frontend - lint cu ESLint și teste cu Vitest.

Regulile de protecție a ramurii main pe GitHub impun ca aceste verificări să treacă înainte ca merge-ul să fie permis. Astfel, cod care nu trece validarea nu poate ajunge niciodată în ramura principală.

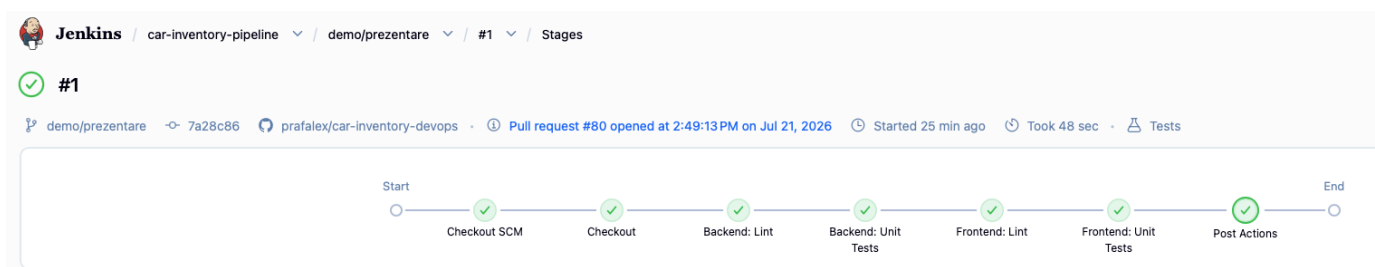


Figura 3: Fluxul car-inventory-pipeline

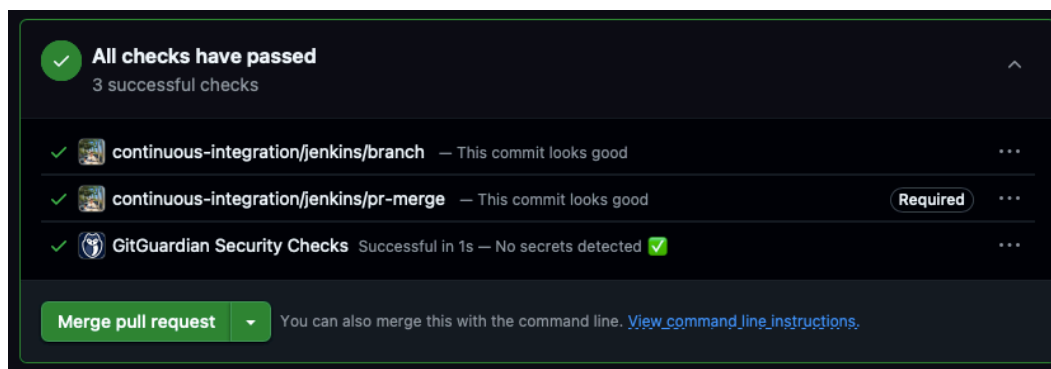


Figura 4: Pull Request din Git ce a trecut de toate verficarile CI

4.2 Pipeline 2 - Staging

Mecanism de declanșare

Pipeline-ul de staging se declanșează automat la fiecare merge în ramura main, prin combinația dintre webhook-ul GitHub și mecanismul de polling SCM. Polling-ul verifică periodic dacă ramura main s-a modificat efectiv și pornește o construcție doar atunci când există un commit nou.

Logica de execuție

- Build imagini - construirea imaginilor Docker pentru frontend și backend, etichetate cu numărul build-ului și cu latest.
- Push în GitHub Packages - publicarea imaginilor în registrul GHCR.
- Deploy pe staging - Jenkins se conectează prin SSH la mașina de staging, transferă fișierul de environment, face pull la cod și pornește containerele cu docker compose.
- Teste de integrare - rularea testelor de integrare direct în containerul backend de pe mașina de staging, prin cereri HTTP reale către aplicația funcțională.

Un aspect important al acestei arhitecturi este că Jenkins nu rulează aplicația local, ci se conectează prin SSH la mașina de staging și rulează comenzile acolo. Jenkins are rolul de orchestrator, iar mașina de staging găzduiește efectiv containerele.

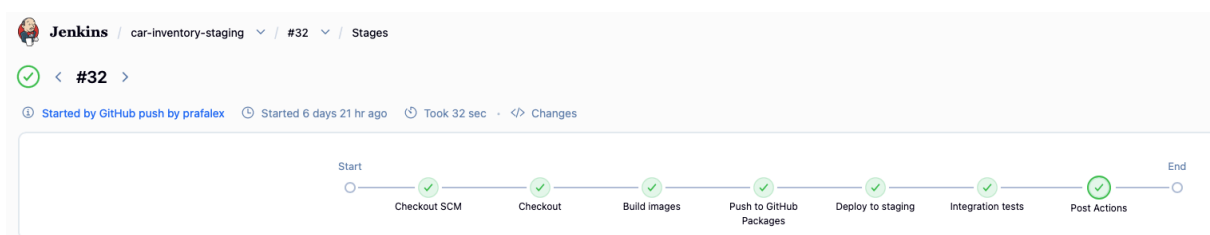


Figura 5: Rulare reușită a pipeline-ului car-inventory-staging (build, push, deploy prin SSH și teste de integrare)

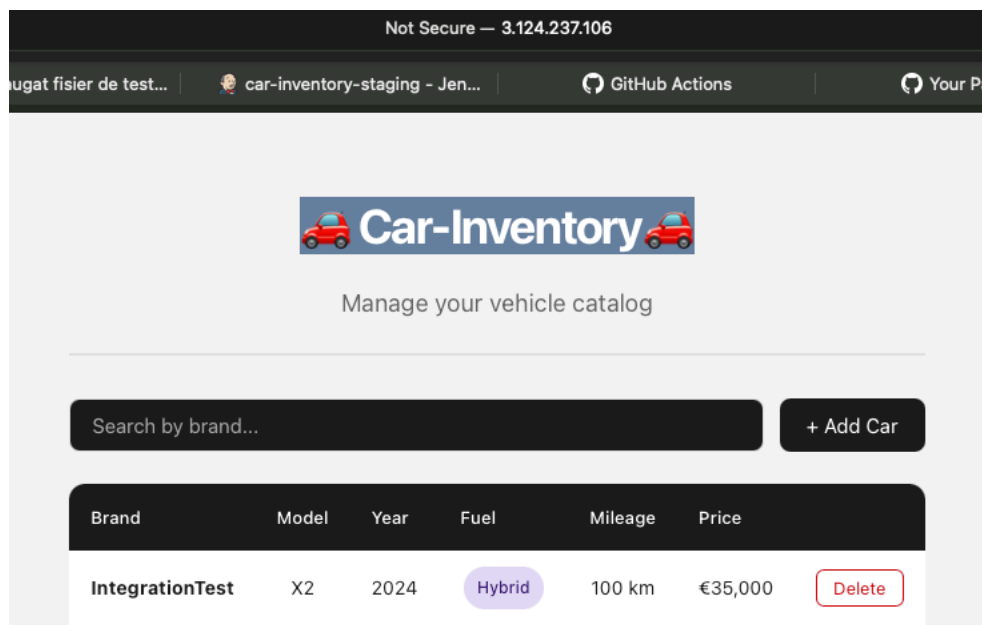


Figura 6: Aplicația de staging accesată în browser la adresa publică a mașinii de staging

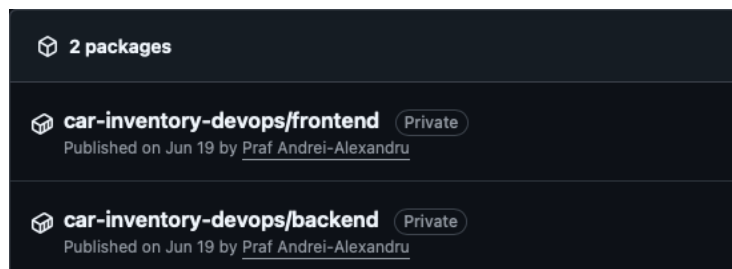


Figura 7: Imaginile Docker publicate în GitHub Container Registry (GHCR)

4.3 Pipeline 3 - Producție (Blue/Green)

Mecanism de declanșare

Pipeline-ul de producție urmărește exclusiv ramura release. Implementarea în producție are loc doar după ce un pull request din main către release este aprobat manual și apoi merge-uit. Această aprobare manuală constituie gate-ul de release: nimic nu ajunge în producție fără o decizie umană explicită, iar fiecare implementare are un pull request corespunzător care servește ca urmă de audit.

Pentru a garanta că pipeline-ul se declanșează doar când ramura release se modifică efectiv, am adăugat o verificare care compară hash-ul commit-ului curent din release cu ultimul commit implementat. Dacă cele două sunt identice, pipeline-ul se oprește imediat, fără a redeploya. Astfel, o modificare doar în main nu declanșează niciodată o implementare inutilă în producție.

Logica de execuție

- Verificare release nou - compararea commit-ului curent cu ultimul implementat; dacă nu s-a schimbat nimic, pipeline-ul se oprește.
- Identificarea mediilor - citirea configurației Nginx de pe mașina de producție pentru a determina care mediu este activ (Blue sau Green) și care este inactiv.
- Backup la baza de date - un `pg_dump` complet cu timestamp, executat înainte de orice implementare, ca punct de restaurare.
- Deploy pe mediul inactiv - implementarea noii versiuni în mediul inactiv, fără a atinge mediul activ care servește traficul.
- Teste smoke - verificarea sănătății noii versiuni în mediul inactiv.
- Comutarea traficului - actualizarea configurației Nginx pentru a direcționa traficul către noul mediu.
- Monitorizare post-comutare - verificări de sănătate repetate; dacă apar probleme, se face rollback automat la mediul anterior.

Strategia Blue/Green și baza de date partajată

Strategia Blue/Green presupune existența a două medii de producție identice. La un moment dat, unul singur este activ și primește trafic, iar celălalt este inactiv, pregătit pentru următoarea implementare. Când se implementează o versiune nouă, aceasta ajunge mai întâi pe mediul inactiv. Doar după ce testele smoke confirmă că totul funcționează, traficul este comutat instantaneu prin Nginx, fără downtime. Dacă apar probleme după comutare, sistemul revine automat la mediul anterior.

Un aspect esențial, corectat pe parcursul proiectului în urma feedback-ului primit, este că cele două medii Blue și Green partajează aceeași bază de date de producție. Acesta este comportamentul corect pentru Blue/Green: datele aparțin aplicației, nu unui anumit mediu. Doar codul diferă între Blue și Green, iar datele rămân constante și partajate. Bazei de date i-am adăugat un backup automat înainte de fiecare implementare, exact pentru că, într-o bază de date partajată, o eroare într-o versiune nouă ar putea afecta datele reale.

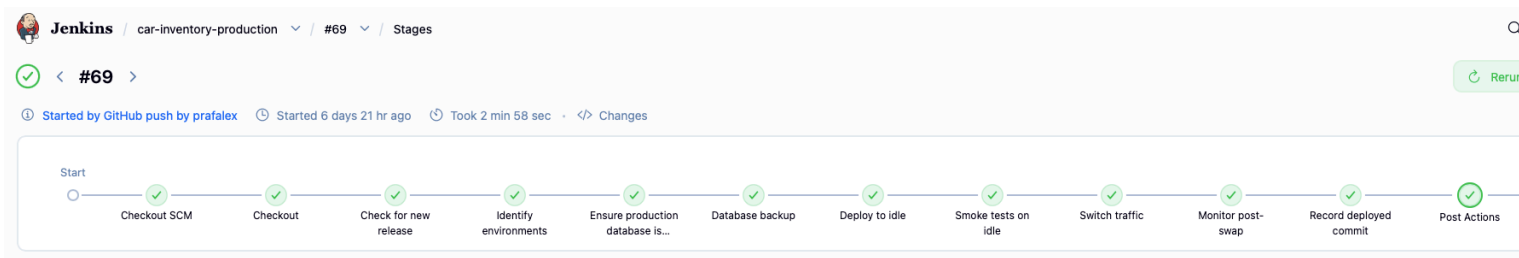


Figura 8: Rulare reușită a pipeline-ului car-inventory-production, cu toate stage-urile Blue/Green

```

~/Desktop/curs_devops/proiect_final/car-inventory-devops % ssh -i ~/.ssh/car-inventory-key.pem ec2-user@63.183.74.202 \
"cat /home/ec2-user/infra-state/nginx-prod.conf"

upstream live_backend {
    server localhost:8003;
}

upstream live_frontend {
    server localhost:3003;
}

server {
    listen 80;

    location /api/ {
        proxy_pass http://live_backend/api/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }

    location / {
        proxy_pass http://live_frontend/;
        proxy_set_header Host $host;
    }
}

```

Figura 9: Fișierul nginx-prod.conf de pe mașina de producție, indicând portul mediului activ

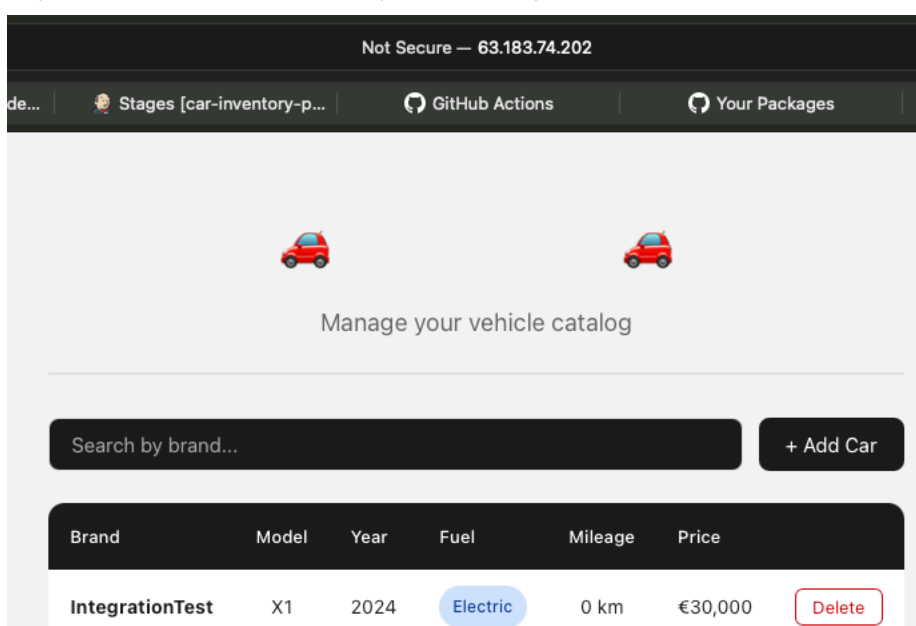


Figura 10: Aplicația de producție accesată în browser la adresa publică a mașinii de producție

4.4 Pipeline 4 - Monitorizare și auto-recuperare

Mecanism de declanșare

Pipeline-ul de monitorizare rulează automat în fiecare minut, printr-un trigger de tip cron definit direct în Jenkinsfile. Nu depinde de modificări de cod, ci de trecerea timpului.

Logica de execuție

- Identificarea mediului activ prin citirea configurației Nginx de pe mașina de producție.
- Verificări de sănătate pentru backend și frontend, executate prin SSH pe mașina de producție.

- Auto-recuperare - după trei eșecuri consecutive ale unui serviciu, containerul afectat este repornit automat, iar contorul de eșecuri este resetat.

Contoarele de eșecuri sunt păstrate pe mașina Jenkins, într-o locație persistentă, astfel încât să supraviețuiască între rulările pipeline-ului. Mecanismul asigură disponibilitatea aplicației fără intervenție manuală: dacă un container cade neașteptat, sistemul îl detectează și îl repornește în câteva minute.

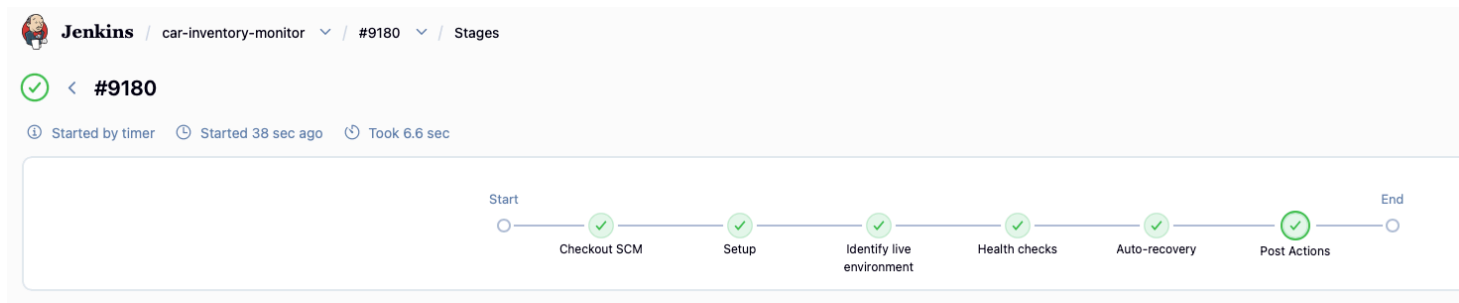


Figura 11: Rulare a pipeline-ului car-inventory-monitor, cu mediul live detectat și status HEALTHY

ANEXE

Anexa 1 - Infrastructura ca și cod (Terraform)

Fragmentul de mai jos ilustrează definirea unei instanțe EC2 în Terraform, cu scriptul `user_data` care configurează automat mașina la prima pornire (instalare Docker, adăugare swap, clonare repository). Întreaga infrastructură (cele 3 VM-uri, grupul de securitate și adresele IP fixe) este definită astfel și poate fi recreată cu o singură comandă.

```
resource "aws_instance" "production" {
  ami              = var.ami_id
  instance_type    = var.production_instance_type
  key_name         = var.key_name
  vpc_security_group_ids = [aws_security_group.car_inventory_sg.id]

  user_data = <<-EOF
  #!/bin/bash
  dnf install -y docker git
  systemctl start docker && systemctl enable docker
  docker network create car-production-network || true
  git clone ${var.github_repo} car-inventory-devops
  ...
  EOF
}
```

Anexa 2 - Reguli de securitate și IP fix

Grupul de securitate controlează traficul de intrare (ingress) și de ieșire (egress). Accesul SSH este restricționat la IP-ul propriu, în timp ce porturile aplicației sunt deschise după nevoie. Fiecărei mașini i se alocă o adresă IP fixă (Elastic IP), care nu se schimbă la repornirea instanței.

```
ingress {
  from_port = 22
  to_port   = 22
  protocol  = "tcp"
  cidr_blocks = [var.my_ip]
}

resource "aws_eip" "production_eip" {
  instance = aws_instance.production.id
  domain   = "vpc"
}
```

Anexa 3 - Verificarea unui release nou (Jenkinsfile.production)

Această logică asigură că pipeline-ul de producție implementează doar atunci când ramura release primește efectiv un commit nou, comparând hash-ul curent cu ultimul commit implementat.

```
stage('Check for new release') {
  steps {
    script {
      env.CURRENT_COMMIT = sh(script: "git rev-parse HEAD",
                              returnStdout: true).trim()
      def lastDeployed = sh(script: "cat ${LAST_COMMIT_FILE} || echo none",
                            returnStdout: true).trim()
      if (env.CURRENT_COMMIT == lastDeployed) {
        env.SKIP_DEPLOY = "true" // nimic nou, se oprește
      }
    }
  }
}
```

```

    } else {
      env.SKIP_DEPLOY = "false" // release nou, se implementează
    }
  }
}
}
}

```

Anexa 4 - Comutarea traficului Blue/Green

Comutarea traficului se realizează prin modificarea configurației Nginx de pe mașina de producție, schimbând porturile din mediul activ în mediul nou implementat.

```

stage('Switch traffic') {
  steps {
    sh """
      ssh -i $SSH_KEY ec2-user@${PROD_HOST} \
        "sed -i 's|localhost:${LIVE_PORT}|localhost:${IDLE_PORT}|g' \
          ${NGINX_CONF}"
    """
  }
}

```

Anexa 5 - Dovezi vizuale suplimentare

☐	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elastic IP
☐	car-inventory-production	i-0b3147bd8be04a221	Running	t3.small	3/3 checks passed	eu-central-1b	ec2-63-183-74-202.eu...	63.183.74.202	63.183.74.202
☐	car-inventory-staging	i-55bb9113b6d48f2	Running	t3.small	3/3 checks passed	eu-central-1b	ec2-3-124-237-106.eu...	3.124.237.106	3.124.237.106
☐	car-inventory-jenkins	i-0000b54dfb9baa247	Running	t3.small	3/3 checks passed	eu-central-1b	ec2-63-183-251-187.eu...	63.183.251.187	63.183.251.187

Figura 12: Consola AWS EC2 cu cele trei instanțe (jenkins, staging, production) în stare Running și Elastic IP-urile alocate

S	W	Name ↓	Last Success	Last Failure	Last Duration
✓	☀	car-inventory-monitor	59 sec #9172	6 days 22 hr #9014	6.2 sec
📄	☀	car-inventory-pipeline	22 min log	N/A	12 sec
✓	☀	car-inventory-production	19 min #70	13 days #17	6.3 sec
✓	☀	car-inventory-staging	9 min 11 sec #33	13 days #13	1 min 25 sec

Figura 13: Dashboard-ul Jenkins cu toate cele patru job-uri, toate cu status verde

☒ **Require a pull request before merging**
Require all commits be made to a non-target branch and submitted via a pull request before they can be merged.

Hide additional settings ^

Required approvals

1 ▾

The number of approving reviews that are required before a pull request can be merged.

☐ **Dismiss stale pull request approvals when new commits are pushed**
New, reviewable commits pushed will dismiss previous pull request review approvals.

☐ **Require review from specific teams**
A collection of reviewers and associated file patterns. Each reviewer has a list of file patterns which determine the files that reviewer is required to review.

☐ **Require review from Code Owners**
Require an approving review in pull requests that modify files that have a designated code owner.

☐ **Require approval of the most recent reviewable push**
Whether the most recent reviewable push must be approved by someone other than the person who pushed it.

☐ **Require conversation resolution before merging**
All conversations on code must be resolved before a pull request can be merged.

Allowed merge methods

Merge, Squash, Rebase ▾

When merging pull requests, you can allow any combination of merge commits, squashing, or rebasing. At least one option must be enabled.

☒ **Require status checks to pass**
Choose which status checks must pass before the ref is updated. When enabled, commits must first be pushed to another ref where the checks pass.

Show additional settings ▾

Figura 14: Regula de protecție a ramurii release din GitHub (require pull request, require approval)